
BIG DATA: Diseño y Arquitectura de Soluciones con Hadoop y Spark

Antonio Soto
CEO
asoto@solidq.com



Laboratorios

- Laboratorio 06 A: Primeros pasos con SPARK
- Laboratorio 06 B Dataframes
- Laboratorio 06 C SparkML
- Laboratorio 06 D Spark Streaming

Laboratorio 06 A: Primeros pasos con SPARK

- Revisando parámetros de configuración
- Contando palabras con SPARK
- Revisando lo que ha pasado

3

Ejercicio 1: Revisando parámetros de configuración

Los tres parámetros clave que se pueden utilizar para la configuración de Spark según los requisitos de la aplicación son **spark.executor.instances**, **spark.executor.cores** y **spark.executor.memory**. Un ejecutor es un proceso que se inicia para una aplicación Spark. Se ejecuta en el nodo de trabajo y es responsable de realizar las tareas de la aplicación. El número predeterminado de ejecutores y el tamaño de estos para cada clúster se calcula en función del número de nodos de trabajo y el tamaño de estos. Esta información se almacena en **spark-defaults.conf (/etc/spark2/2.6.5.3003-25/0)** en los nodos principales del clúster. Los tres parámetros de configuración se pueden configurar en el nivel de clúster (para todas las aplicaciones que se ejecutan en el clúster) o se pueden especificar también para cada aplicación individual.

Es habitual que dependiendo del trabajo a realizar estas configuraciones se modifican para un trabajo en concreto en tiempo de ejecución, en lugar de modificarlas para todo el cluster. Vamos a ver como hacerlo desde un Notebook de Jupyter. Para ello:

- Iniciamos el entorno de JUPYTER bien desde el portal de Azure o bien

directamente en el navegador: [https://NOMBRECLUSTER/ JUPYTER](https://NOMBRECLUSTER/JUPYTER)

- Creamos una nueva carpeta llamada Curso
- Dentro de la carpeta cargamos el fichero Laboratorio 08 A Primeros paso con SPARK
- Insertamos una celda de tipo Code como primera celda y especificamos

```
%%configure
```

```
{"executorMemory": "3072M", "executorCores": 4, "numExecutors":10}
```

- Ejecutamos la celda y nos mostrará el cambio de configuración. Si quisiésemos cambiar la configuración una vez iniciada la sesión Spark, deberíamos de especificar el parámetro -f

Ejercicio 2: Cuenta palabras con SPARK

La segunda celda del Notebook nos muestra como cargar ficheros de texto y manipularlos desde Spark. El contexto que tenemos cargado de forma predeterminada utiliza YARN como modo de ejecución. Ejecutamos la segunda celda revisando las operaciones que va realizando en cada línea.

Ejercicio 3: Revisando lo que ha pasado

Si haces click en el enlace mostrado de Spark UI cuando se ha generado la aplicación podrás acceder al entorno donde visualizar que es lo que Spark ha hecho para ejecutar ese código.

¿Cuántas etapas tiene el job? ¿Cuántas tareas tiene cada etapa?

Laboratorio 06 B DataFrames

- Cargar datos de texto
- Preparar esquema para la carga
- Join entre tablas
- Consultas HIVE

4

Ejercicio 3: DataFrames

En este ejercicio, verás como trabajar con DataFrames importando datos desde ficheros, en este caso, csv.

Verás también como aplicar transformaciones y filtros sobre esos datos, así como la creación de estructuras en memoria que te permitan después utilizar sentencias SELECT para la consulta de esos datos.

Finalizarás el ejercicio viendo la interacción entre SPARKSQL y HIVE, almacenando los datos en una tabla y consultando tablas HIVE desde SPARK.

Vete ejecutando las celdas desde la cabecera de DataFrames.

Laboratorio 06 C SparkML

- Definición de Transformador
- Definición de Estimador
- Creando el pipeline

5

Ejercicio 1: Machine Learning

En este laboratorio veremos como crear, entrenar y consultar un modelo predictivo con Spark

Los datos siguientes muestran la temperatura objetivo y la temperatura real de algunos edificios con sistemas HVAC instalados. La columna System representa el identificador del sistema y la columna SystemAge, el número de años que lleva el sistema HVAC instalado en el edificio. Los datos de este tutorial se usan para predecir si un edificio será más cálido o frío en función de la temperatura objetivo, dados un identificador del sistema y la antigüedad del sistema

Cargaremos los datos del .csv. A través de una función que compara la temperatura real con la temperatura de destino indicaremos que si la temperatura real es mayor, el edificio está **cálido**, lo que viene indicado por el valor 1.0. De lo contrario, el edificio está **frío**, lo que se indica con el valor 0.0.

Generaremos un **pipeline** de Spark, utilizando un algoritmo de Regresión Logística.

Un **pipeline** es un flujo de trabajo completo que combina varios algoritmos de ML. Los pipelines definen las fases y el orden de un proceso de ML.

Un **transformador** es un algoritmo que transforma un elemento DataFrame en otro. Por ejemplo, un transformador de características puede leer una columna de un elemento DataFrame, asignarla a otra columna y generar un nuevo elemento DataFrame con la columna asignada anexada

Un **estimador** es una abstracción de algoritmos de aprendizaje y es responsable del ajuste o el aprendizaje en un conjunto de datos para producir un transformado

En nuestro caso generaremos un pipeline que consta de tres fases: tokenizer, hashingTF (ambos son transformadores) y lr:

1. En la primera fase, Tokenizer, se divide la columna de entrada SystemInfo (que consta de los valores de identificador y antigüedad del sistema) en una columna de salida words. Esta nueva columna words se agrega a DataFrame
2. En la segunda fase, HashingTF, se convierte la nueva columna words en vectores de característica. Esta nueva columna features se agrega al elemento DataFrame. Estas dos primeras fases son transformadores.
3. La tercera fase, LogisticRegression, es un estimador, por lo que la canalización llama al método LogisticRegression.fit() para generar un modelo LogisticRegressionModel.

Prepararemos después unos datos de prueba para probar el modelo y veremos sus predicciones.

Para todo ello, vete paso a paso a través del laboratorio

Laboratorio 06 D Spark Streaming

- Spark Streaming
- Streaming Estructurado

6

Ejercicio 1: SPARK Streaming

Veremos en este ejercicio las bases de SPARK Streaming para crear aplicaciones que sean capaces de analizar datos en tiempo real a través de streams. Carga para ello el notebook asociado y ejecuta las instrucciones como se te indican en el Notebook

En un escenario más real, lo habitual es utilizar un streaming estructurado que acabe generando tablas o ficheros para un consumo posterior por parte de ETL o directamente herramientas de BI. Para ello revisa el Notebook



www.solidq.com

info@solidq.com